

# MACHINE LEARNING

## BOOSTING

Sebastian Engelke

MASTER IN BUSINESS ANALYTICS



**UNIVERSITÉ  
DE GENÈVE**

# Boosting for trees

- ▶ The idea of **bagging** is to use many **unbiased** models with **high variance** and reduce the variance by **averaging**.
- ▶ Bagging involved fitting **many large trees** to the bootstrap data sets to create a single predictive model.

# Boosting for trees

- ▶ The idea of **bagging** is to use many **unbiased** models with **high variance** and reduce the variance by **averaging**.
- ▶ Bagging involved fitting **many large trees** to the bootstrap data sets to create a single predictive model.
- ▶ Like bagging, **boosting** is a general approach that can be applied to many statistical learning methods for **regression or classification**. We only discuss boosting for **regression trees**.
- ▶ Boosting works in a similar way as bagging, but the principle is to use many **weak, biased** models with **low variance**, and combine them to **reduce the bias**.

# Boosting for trees

- ▶ The idea of **bagging** is to use many **unbiased** models with **high variance** and reduce the variance by **averaging**.
- ▶ Bagging involved fitting **many large trees** to the bootstrap data sets to create a single predictive model.
- ▶ Like bagging, **boosting** is a general approach that can be applied to many statistical learning methods for **regression or classification**. We only discuss boosting for **regression trees**.
- ▶ Boosting works in a similar way as bagging, but the principle is to use many **weak, biased** models with **low variance**, and combine them to **reduce the bias**.
- ▶ The **main difference** is that in boosting, the trees are **grown sequentially**: each tree is grown using information from previously grown trees.

## Boosting algorithm for regression trees

Fix the following constants:

- (a) The number of **repetitions**  $B \in \mathbb{N}$ .
- (b) The number of **splits**  $d \in \mathbb{N}$  in each tree (i.e.,  $d + 1$  terminal nodes).
- (c) The **shrinkage** parameter  $\lambda \in (0, 1)$ .

## Boosting algorithm for regression trees

Fix the following constants:

- (a) The number of **repetitions**  $B \in \mathbb{N}$ .
- (b) The number of **splits**  $d \in \mathbb{N}$  in each tree (i.e.,  $d + 1$  terminal nodes).
- (c) The **shrinkage** parameter  $\lambda \in (0, 1)$ .

Then run the following algorithm on the training set  $(x_i, y_i)$ ,  $i = 1, \dots, n$ :

- (1) Set  $\hat{f}(x) = 0$  and  $r_i = y_i$  for all  $i = 1, \dots, n$ .
- (2) For  $b = 1, \dots, B$  repeat:
  - (i) Fit a tree  $\hat{f}^b$  with  $d$  splits to the temporary training data  $(x_i, r_i)$ ,  $i = 1, \dots, n$ .
  - (ii) Update the predictive model by adding a **shrunk version** of the new tree

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

- (iii) Update the residuals

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

- (3) Output the **boosted model**

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x).$$

## Boosting for trees

- ▶ Boosting is a form of **slow learning**, since we are fitting small trees that are slowly improving  $\hat{f}$  in areas where it does not perform well.
- ▶ The shrinkage parameter  $\lambda$  slows the process down even further, allowing **more and different shaped** trees to attack the residuals.

## Boosting for trees

- ▶ Boosting is a form of **slow learning**, since we are fitting small trees that are slowly improving  $\hat{f}$  in areas where it does not perform well.
- ▶ The shrinkage parameter  $\lambda$  slows the process down even further, allowing **more and different shaped** trees to attack the residuals.

Boosting has three **tuning parameters**:

- (a) The number of trees  $B$ . Unlike bagging and random forests, boosting can **overfit** if  $B$  is too large, although this overfitting tends to occur slowly. We use **CV** to select  $B$ .



# Boosting for trees

- ▶ Boosting is a form of **slow learning**, since we are fitting small trees that are slowly improving  $\hat{f}$  in areas where it does not perform well.
- ▶ The shrinkage parameter  $\lambda$  slows the process down even further, allowing **more and different shaped** trees to attack the residuals.

Boosting has three **tuning parameters**:

- (a) The number of trees  $B$ . Unlike bagging and random forests, boosting can **overfit** if  $B$  is too large, although this overfitting tends to occur slowly. We use **CV** to select  $B$ .
- (b) The shrinkage parameter  $\lambda$  is typically a small positive number such as 0.01 or 0.001. It controls the **learning rate** of boosting. Smaller values may require larger  $B$  for good performance.

# Boosting for trees

- ▶ Boosting is a form of **slow learning**, since we are fitting small trees that are slowly improving  $\hat{f}$  in areas where it does not perform well.
- ▶ The shrinkage parameter  $\lambda$  slows the process down even further, allowing **more and different shaped** trees to attack the residuals.

Boosting has three **tuning parameters**:

- (a) The number of trees  $B$ . Unlike bagging and random forests, boosting can **overfit** if  $B$  is too large, although this overfitting tends to occur slowly. We use **CV** to select  $B$ .
- (b) The shrinkage parameter  $\lambda$  is typically a small positive number such as 0.01 or 0.001. It controls the **learning rate** of boosting. Smaller values may require larger  $B$  for good performance.
- (c) The number  $d$  of splits in each tree controls the **complexity** of the boosted ensemble. Often  $d = 1$  works well, in which case each tree is a **stump**, consisting of a single split.

# Boosting for trees

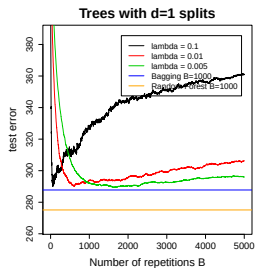
- ▶ Boosting is a form of **slow learning**, since we are fitting small trees that are slowly improving  $\hat{f}$  in areas where it does not perform well.
- ▶ The shrinkage parameter  $\lambda$  slows the process down even further, allowing **more and different shaped** trees to attack the residuals.

Boosting has three **tuning parameters**:

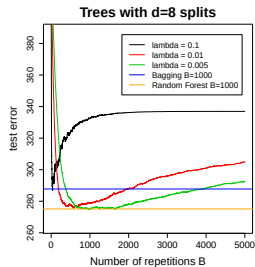
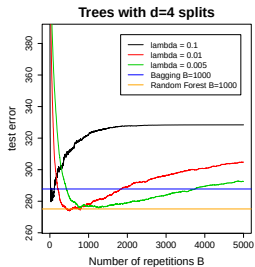
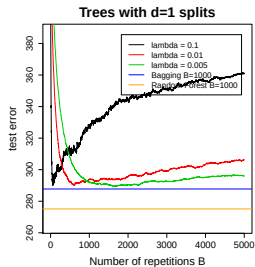
- (a) The number of trees  $B$ . Unlike bagging and random forests, boosting can **overfit** if  $B$  is too large, although this overfitting tends to occur slowly. We use **CV** to select  $B$ .
- (b) The shrinkage parameter  $\lambda$  is typically a small positive number such as 0.01 or 0.001. It controls the **learning rate** of boosting. Smaller values may require larger  $B$  for good performance.
- (c) The number  $d$  of splits in each tree controls the **complexity** of the boosted ensemble. Often  $d = 1$  works well, in which case each tree is a **stump**, consisting of a single split.

**Boosting for classification** is similar but a bit more complex to formulate.

# Hitters data: boosting



# Hitters data: boosting



# Bagging vs Boosting